# Genie for the Business Workshop

Author: Barry Hodges, Databricks Senior Solution Architect, New Zealand

Version: 1

Date: 14-Jul-2026

# Contents

# Introduction

Genie Code is an autonomous AI partner purpose-built for data work in Databricks. Unlike other AI assistants, Genie Code is deeply integrated with Databricks capabilities like Unity Catalog, allowing it to understand your complete data landscape, including tables, columns, and lineage. This contextual awareness enables Genie Code to accelerate complex data workflows while autonomously adapting to your specific data and governance model. Genie Code is designed for data teams to use every day, from experimentation and model development to production pipelines and BI dashboards. It is available throughout the Databricks Lakehouse, and chat threads persist as you navigate between pages. These hands-on labs will guide you through Genie Code's core capabilities using a Databricks Free Edition workspace.

# What you will cover

- **Lab 1: Getting Started** - Get familiar with Genie Code basics.
- **Lab 2: Lakeflow Designer** - Build a simple no-code ETL pipeline using the Lakeflow Designer canvas and prompts.
- **Lab 3: Data Analysis and Visualisation** - Create an AI/BI Dashboard from the Lab 2 table - all driven by natural language.
- **Lab 4: Genie Agents** - Build a Genie Agent from the Lab 2 table to support natural language analysis.
- **Lab 5: Exploratory Data Analysis** - Let Genie Code autonomously plan and execute a full exploratory data analysis (EDA).

**** IMPORTANT ****

- You can complete the labs in this guide without prior Databricks experience, but a basic understanding of Databricks concepts such as Notebooks, Pipelines, Unity Catalog, Dashboards, etc. will help you better appreciate Genie Code's value.
- It is recommended you complete the labs in order as the knowledge you build in earlier labs helps with the later labs. Additionally the table you create in Lab 2 is a prerequisite to Labs 3, 4 and 5.
- You don't need to complete every lab in a single sitting. These labs run on the Free Edition - true to its name, it remains free - so you can progress at your own pace.
- Instructions in earlier labs are in some cases more verbose than later labs - the assumption is you don't need every specific detail as you move through labs. For example, at the start of some of the labs there is a check list of steps you must complete before proceeding (**similar** to the following). **Missing any means you will almost certainly run into problems**.

  **'+ New'** → **'Notebook'** ✅, Default language **'SQL'** ✅, Genie Code pane open ✅, **'New Chat'** ✅

# Working with Free Edition and Genie Code

- **Free Edition shares back-end resources** and has daily serverless compute quotas and other limitations, so expect it to be slower than production Databricks and potentially throttled (most quotas reset the next day). But hey, it's free 🙂.
- **Genie Code is non-deterministic** - your results may vary. So:
    - Always **review generated code** before executing - Genie Code is powerful but can make mistakes.
    - **Be specific** - more context = better output. Include column names, expected formats, and desired visualisation types.
    - **Work in one chat per lab** - context compounds, and Genie Code gets better at predicting what you want as the conversation grows.
    - **If Genie Code goes off-track** - stop it (red square button) and re-prompt rather than letting it build on a bad foundation.
    - **Most importantly, play** - write your own prompts to explore. If you get errors, work with Genie Code to resolve.
- Genie Code can generate and run code in your notebook. While it has guardrails to prevent dangerous actions and respects the user's Unity Catalog permissions, there is still risk. Use it only with code and data you trust.
- **Approvals and controls** - Genie Code often pauses for your input as it works. For more complex tasks it may first lay out a step-by-step plan and ask clarifying questions - answer these to help it refine the plan.

  Before running any code it asks for your approval: click '**Allow**', '**Skip**', '**Decline**', '**Continue**', or '**Reject**' as appropriate. To cut down on prompts you can choose '**Allow in this thread**' or '**Always allow**', or turn on **Auto-approve** - an AI classifier that checks each proposed action against your stated intent and allows or blocks it automatically, minimising manual approvals while still stopping risky ones.

  For these labs we recommend clicking '**Allow**' one step at a time so you can see exactly what Genie Code is doing. To stop it mid-task, click the stop button ■.
- Genie Code is a **rapidly evolving** technology. The experience and features may change (for the better) when you next try these labs.

# TPC-H Schema

Fee Edition comes with several sample data sets. In this lab you will be using data in the samples.tpch schema. TPC-H is a widely used decision-support benchmark from the Transaction Processing Performance Council that models a global wholesale-distribution business: customers place orders, each order has one or more line items, and line items are for parts supplied by suppliers; customers and suppliers belong to nations, and nations belong to regions.

It has eight tables:

- region - the 5 geographic regions (e.g. AMERICA, EUROPE, ASIA)
- nation - 25 nations, each belonging to a region
- customer - customers, each in a nation, with a market segment (e.g. BUILDING, AUTOMOBILE)
- supplier - suppliers, each in a nation
- part - the parts / products that can be ordered
- partsupp - which supplier supplies which part (available quantity, supply cost)
- orders - customer orders (order date, priority, status, total price)
- lineitem - the individual lines on each order (part, supplier, quantity, price, discount, ship dates)

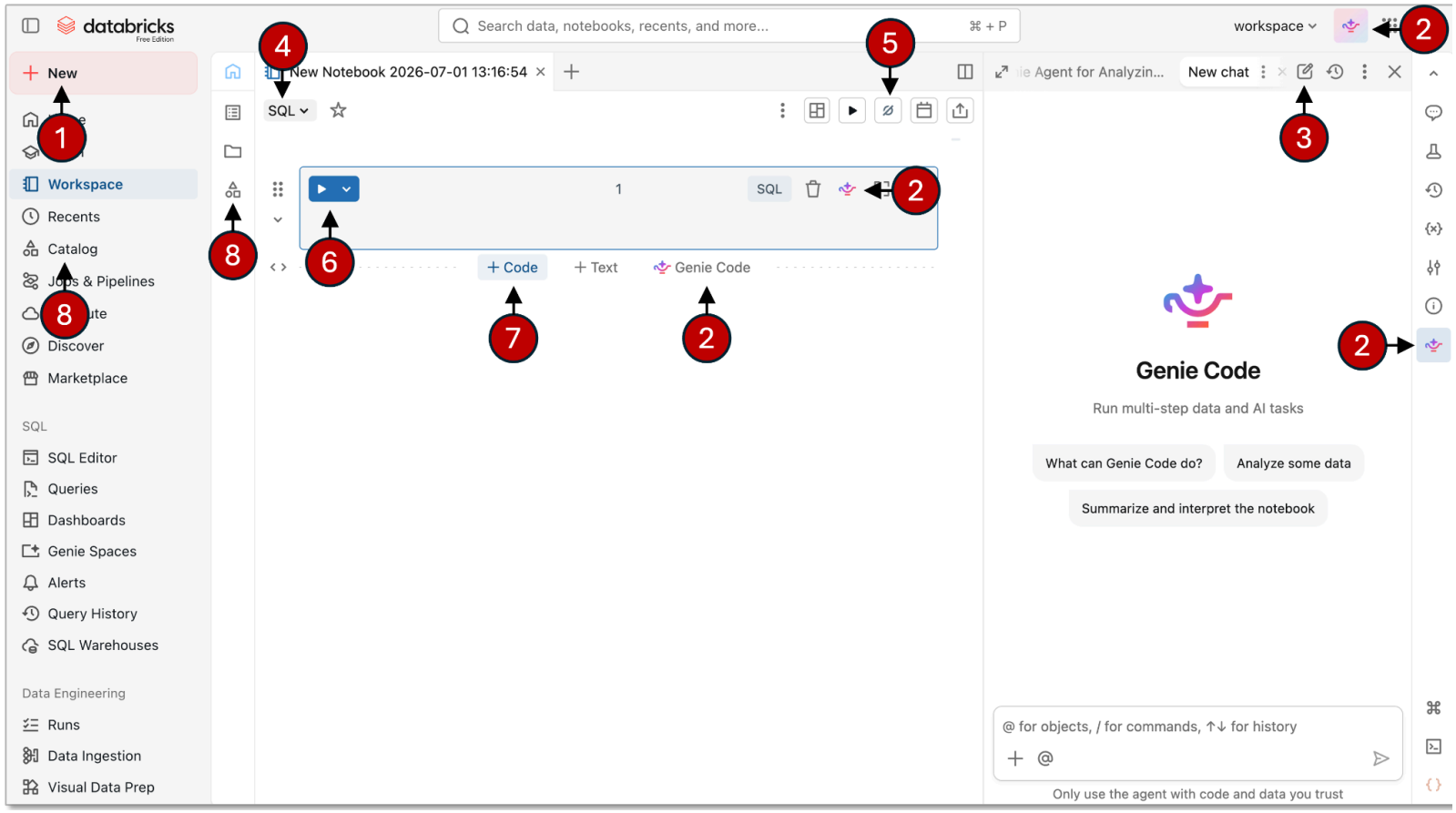
These join through natural keys (region → nation → customer / supplier, and customer → orders → lineitem → part / supplier) which makes the dataset excellent for practising joins, aggregations and analytics. The data is synthetic and its dates are from the 1990s; that age has no bearing on its value here - the clean relational structure and realistic business questions are exactly what these labs need.

# Resources:

- Databricks Free Edition Documentation
- Genie Code Documentation (AWS, Azure, GCP)
- Introducing Genie Code - launch announcement and feature overview
- Get to Know Genie Code - demo covering pipelines and dashboards
- Databricks YouTube Channel (filtered by 'Genie Code') - tutorials and demos
- Databricks Genie Code: Full Practice Guide and Examples
- From Genie to Genie Code: Agentic Data Work on Databricks
- Agentic Data Engineering with Genie Code and Lakeflow

# The Databricks Workspace UI

After clicking '+ New', 'Notebook'




1. Click to create a new Notebook / ETL Pipeline / etc.
2. Bring up the Genie Code panel
3. Start a new chat
4. Toggle Notebook default language between **Python** and **SQL**
5. Select compute type (**Serverless** or **Serverless Starter Warehouse**)
6. Run Cell
7. Click '**+ Code**' to add an empty cell below the current cell
8. Unity Catalog - explore data related objects

# Setup

**Objective:** Setup your Databricks Free Edition workspace; Create a schema where objects such as tables will be created.

---

1. If you haven't already, sign up for **Databricks Free Edition** at www.databricks.com/learn/free-edition (work or personal email).
2. Once connected to your Databricks Free Edition workspace, click on the **Genie Code** icon in the upper-right corner .
3. In the chat pane, enter the following:

   > Create a new schema workspace.genie_demo.

4. When prompted, click '**Allow**' so Genie Code can run the generated code and create the schema which is used by the labs.
5. To confirm the schema exists, in the left hand menu, click on '**Catalog**' which opens the Unity Catalog explorer.
6. Now click on the catalog '**workspace**'. You should see the schema '**genie_demo**' (as well as '**default**' and '**information_schema**').

---

# Lab 1: Getting Started

**Objective:** Learn to interact with Genie Code; Explore the UI; Use @ table references.

---

1. **'+ New' → 'Notebook'** ✅, Default language **'SQL'** ✅, Genie Code pane open ✅, **'New Chat'** ✅
2. In the chat pane, type the following:

   > What tables are available in the samples.tpch schema? Describe each table and how they relate to each other.

3. Genie may ask to run code, potentially multiple times. During these labs we recommend you accept the default '**Ask every time**' and click '**Allow**'.
4. Review the final response. Genie Code uses Unity Catalog metadata to describe the tables and their relationships.

5. Use the @ reference to ask a specific question about a specific table (make sure orders resolves to samples.tpch.orders):

   > How many columns does @orders have? What does each column represent?

   Note: Here you located samples.tpch.orders using @. In the rest of these labs we do not use @ as the tables we are working with are fully qualified tables (i.e. catalog.schema.table) meaning Genie Code knows exactly what table we are interested in.

6. Next, ask Genie Code to generate a simple query:

   > Write a SQL query that shows the top 10 customers by total order value from samples.tpch.orders and samples.tpch.customer.

7. Genie Code should generate the SQL code in the empty cell on the notebook that existed when you first opened the notebook. When prompted to run the cell, click '**Allow**' in the Genie Code pane.

   You may also see '**Run suggested**', '**Reject**', and '**Accept**' buttons in the notebook cell - ignore these as in most cases you will drive code execution via Genie Code.

   From time to time you may see '**Accept all**' at the bottom of the Genie Code pane. Feel free to click this - it will clear any outstanding prompts on the Notebook cells (such as '**Accept**').

8. After clicking '**Allow**' you should see the cell status switch to 'running'. Once the query has completed, you should see the query result set displayed at the bottom of the cell.

9. Try the /explain command. Either:

   - click the Genie icon at the top of the SQL cell and type /explain
   - any where in the SQL cell, press '**Cmd+I**' (Mac) or '**Ctrl+I**' (Windows) and type /explain
   - in the Genie Code pane type /explain

10. Note there are several other / commands you can use with Genie Code including (but not limited to):

    - /findTables - searches for relevant tables based on Unity Catalog metadata
    - /getTableLineages - view upstream and downstream dependencies.
    - /getTableInsights - access metadata-driven insights, such as user activity and query patterns.

11. Genie Code can also explain concepts. Enter the following question:

> What is a window function in Spark? How does it differ from a regular GROUP BY? When would I use one over the other? Just explain this to me, don't run any code.

12. Follow up with:

> What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()? Show me an example using samples.tpch.orders where the difference matters - where the three functions produce different results.

13. Genie Code may add a new Notebook cell below the existing one with the example code or the code may appear in the Genie Code pane in which case:

    - create a new Notebook cell - move the mouse below the last cell until '- - - **+Code** **+Text** **Genie Code** - - -' pops up - click on '**+Code**'.
    - use the '**<<**' or the **Copy code** button at the top of the code snippet in the Genie Code pane to insert it / or click on the copy icon and paste into the new Notebook cell.

    Once the code has been entered into the new cell, run the sql statement by:

    - either click '**Allow**' in the Genie Code pane
    - or click the '►' run cell button top left of the SQL cell.

    Verify you can see where the three functions diverge.

---

# Lab 2: Data Workflow with Lakeflow Designer

**Objective:** Build a no-code ETL pipeline in Lakeflow Designer that uploads a CSV file, joins to a other data sources from Unity Catalog, and finally writes the result table orders_enriched. Along the way you will apply data quality transformations and filters.

**New in this lab:** Lakeflow Designer is a visual, no-code pipeline builder for ETL on Databricks. It uses a drag-and-drop canvas and / or Genie Code, so a business analyst can build a production-grade pipeline without writing code. Behind the scenes, every Designer pipeline compiles down to a Lakeflow Spark Declarative Pipeline - so it gets the same governance (Unity Catalog), observability, scheduling and reliability as a hand-coded pipeline. See Announcing Lakeflow Designer / Lakeflow Pipelines Editor.

1. **'+ New' → 'Notebook'** ✅, Default language **'SQL'** ✅
2. Paste the following SQL statement into the empty cell at the top of the Notebook - it will select 1,000 random records from the samples.tpch.orders table (which contains 750,000 records):

```
SELECT *
FROM samples.tpch.orders
ORDER BY RAND()
LIMIT 1000;
```

3. In the result table, click the download icon ( ⌄ ) at the bottom-left of the result grid and choose **'Download CSV'.** Save the file as **'orders_recent.csv'** somewhere on your laptop where it will be easy to find.
4. Click **'+ New' → 'Visual data prep'**. The Lakeflow Designer canvas opens.
5. Click **'Upload a file'**, click on **'browse'**, **'select files'** and select **'orders_recent.csv'** from where you saved the file on your laptop.
6. Under '**Destination volume**', click **'+ Create volume'**.
    - Volume name = **'uploads'**,
    - Volume type = **'Managed volume'**
    - Catalog = **'workspace'**
    - Schema = **'genie_demo'**
7. Click **'Create'** and then **'Upload'**.
8. Once the upload finishes a new source node appears on the canvas, connected to the uploaded **'orders_recent.csv'** file (which is now stored in the managed volume you created above - volumes are for storing semi and unstructured files in Unity Catalog).
9. In the Preview section below, scan the columns and notice 'o_totalprice' ranges across small and large orders. We'll filter out orders below 50,000.
10. Before creating the filter, note:
    - Unlike the previous lab where you used the Genie Code pane on the right, this time you will use the option of using the Genie Code pane at the bottom of the Designer canvas. If the right-hand Genie Code pane is still open, please close it to maximise your workspace view.

- Whilst this lab uses Genie Code to do the work, you can do everything yourself by dragging operators from the left menu onto the Designer canvas and / or by right clicking on the canvas and selecting the required operator from there, in which case you will also need to configure each operator as needed via the operator edit button ( ).

11. In the Genie Code pane on the Designer canvas, enter the following prompt:

> Remove rows where o_totalprice is less than 50000.

12. Genie Code should create a new operator connected downstream from the source node. Click '**Accept All**'.

13. At any time you can recentre the canvas. Right-click on the canvas and click on the '**Fit view**' button ( ) or select it from the top left of the canvas. You can also "tidy" up the canvas via the '**Auto layout'** button ( ). Reuse these capabilities as necessary as you go through the rest of this lab.

14. Click the filter operator, then the operator edit button ( ). This will open the editing pane on the right where you can see an expression being applied to the source data similar to `o_totalprice is greater than or equal to "50000"`.

15. Sort the input preview at the bottom left by `o_totalprice` ascending and confirm there are multiple values less than 50,000. The row count (at the very bottom) should show 1,000.

16. Now sort the output preview at the bottom right by `o_totalprice` ascending and confirm the smallest value is at least 50,000 (most likley greater than 50,000). The row count (at the very bottom) should be less than 1,000.

17. Add the customer, nation and region dimension tables as additional sources. Using Genie Code, prompt:

> Add samples.tpch.customer, samples.tpch.nation and samples.tpch.region as sources.

18. Click '**Accept All**'.

19. Join the customer, nation and region tables to bring the nation and region names through (instead of just the IDs). Also change the case so they have an initial upper case letter, followed by lower case letters.

> Join the customer, nation and region tables to bring the nation name and region name through. Also change the nation name and region name case so it has an initial upper case letter, followed by lower case letters.

20. Ask Genie Code to join the two datasets:

> Join the orders source with the customer source. Every order should show the customer details.

21. Click '**Accept All**'.

22. Click the Join node, then the operator edit button ( ✎ ). Note:

    - The join keys (`o_custkey` = `c_custkey`).
    - The join type (`Left join`) - If you don't see '`Left join`', click '`Left join`'below and then click '**Apply**' (see the description of what a left join is. Click on the other join types to compare their behaviour).
    - In the Output you will see almost all customer fields have null values. This is expected in Preview mode, which by default limits all input datasets to 1,000 records. Whilst we limited the orders to 1,000 random records, we need access to all 750,000 customer data records to ensure there is a join. When you run the pipeline later in the lab, it will process all 750,000 customer records, allowing each order to be matched to the correct customer and replacing the null values with the appropriate customer data. You will verify this towards the end of the lab.

23. Ask Genie Code to write the results to a table:

    > Write the joined orders and customer data to a table orders_enriched in the schema workspace.genie_demo.

24. A new Output operator should appear. Click '**Accept All**'.

25. Click on the Output operator which is designed to create a table based on the data being passed to it from the upstream operator. Note you have options to '**Create or replace**', '**Merge**' or '**Append**' ('**Create or replace**' should be selected given the table does not yet exist). You also have the option to change the destination catalog, schema and the table name.

26. Click '**Run**' at the bottom right of the operator editor. This should create and load the table orders_enriched in the workspace.genie_demo catalog/schema.

27. To confirm the table has been correctly created, in the left hand menu, click on '**Catalog**' - this opens the Unity Catalog explorer.

28. Now click on the catalog '**workspace**' (which is your catalog), then '**genie_demo**' (which is your schema).

29. You should see your table - **orders_enriched**.

30. Click on **orders_enriched** and then click on '**Sample Data**' (you may have to select and start your Compute).

31. Scroll to the right to confirm each

    - order has associated customer data

- the Nation / Region names have come through
- the mixed case has been correctly applied

32. The Sample Data view only shows the first 100 rows. You see the total number of rows by clicking on '**Details**' and seeing the value in the '**Properties**' section beside '**sql.statistics.numRows:**' or you can run the following query in the SQL Editor (accessed via the menu on the left):

```
SELECT count(*)
  FROM workspace.genie_demo.orders_enriched;
```

33. Back on the table view in Unity Catalog, click '**Lineage**', '**See lineage graph**' to see the associated python notebook (which always opens in Lakeflow Designer) and other related data objects (e.g. orders_recent.csv file in the volume and the customers table in the samples.tpch schema).

---

# Lab 3: Data Analysis and Visualisation

Objective: Use Genie Code within AI/BI Dashboards to build an interactive business dashboard on the orders_enriched table you created in Lab 2 to show the breadth of dashboard visualisations. You describe each chart in plain business terms and Genie Code finds the data and builds the dataset and widget on the canvas. This lab contains all the prompts you need.

**New in this lab:** Databricks AI/BI Dashboards are governed, low-code dashboards built natively on Unity Catalog. Each dashboard is composed of datasets (SQL queries against your tables) and widgets (the visualisations that read from those datasets). Genie Code can author both - you describe the chart you want and it produces the dataset SQL plus the widget on the canvas. When you publish, the dashboard becomes available to viewers and Databricks automatically creates a companion Genie Agent, letting end users ask follow-up questions of the same data in natural language. See Databricks AI/BI, Genie Agents.

---

1. **'+ New'** → **'Dashboard'** (a blank AI/BI Dashboard will open) ✅, Genie Code pane open ✅, **'New Chat'** ✅
2. In this lab you are back to using the primary Genie Code pane (accessed via the Genie Code icon at the top right).
3. What follows are multiple prompts you can use to create foundation datasets and associated widget based visualisations.

   Run one at a time and observe Genie Code building the Dashboard dataset and associated visualisations on your behalf. Click on each generated visualisation widget and note how it has been configured (see the widget editor on the right). Most widgets will have a dataset

dependency which maps to a dataset listed under the '**Data**' tab at the top left of the Dashboard canvas. These datasets are runtime SQL queries Genie Code has created to provide the requisite data.

If Genie Code does not create what you are expecting to see, use your own prompts to get what you want:

> This dashboard is based on the workspace.genie_demo.orders_enriched table. Add a region filter at the top. As you add datasets and widgets, make sure the filter will work with each of them.

> Add a counter row showing the total number of orders, the total order value, and the average order value. Round values appropriately. Give each a suitable title.

> Add a widget showing the distribution of raw order values by market segment as a box plot. Title it "Order Value Distribution".

> Add a widget showing the average order value by customer market segment as a bar chart. Title it "Average Order Value by Segment".

> Add a widget showing total order value by country as a map titled "Order Value by Country".

> Add a widget showing the top 3 customers by total order value, by region. Title it "Top Customers by Region".

4. Ask Genie Code to suggest additional visualisations / get Genie Code to add any you like the sound of:

> What additions to this dashboard do you suggest?

5. Feel free to ask Genie Code to add any of the recommended additions.
6. Finally, review the dashboard layout. Ask Genie Code to tidy it up:

> Give the dashboard tab and the dashboard itself intuitive names and review the layout to see if you can improve it in any way. Also, use colour more effectively.

7. Click on the '**Data**' tab (top left of the Dashboard) to see all the datasets Genie Code generated to support these visualization widgets (in each case created from a SQL dataset).
8. Click '**Publish**' (top right corner), accept defaults for subsequent prompts.
9. Close the Genie Code pane and click '**View Published**' (top middle) to see the Dashboard as end users will see it.

# Lab 4: Genie Agents - Talk to your Data

**Objective:** Create a Genie Agent over your orders_enriched table, curate it with a few business definitions, and have a natural-language conversation based on the table's data - the same experience you would hand to a non-technical stakeholder.

**New in this lab**: Genie Agents (cf. Genie Code - don't confuse them) are a business-user-facing surface: a curated chat interface published over one or more Unity Catalog tables, designed to be used by non-technical stakeholders. You shape Genie Agent behaviour by adding Instructions (business definitions) and using benchmarks to improve the Genie Agent's quality. **This lab only scratches the surface on what is possible in terms of setting up a Genie Agent**. See Genie Agents documentation.

**Also new:** Genie One is Genie's single, business-user entry point for data questions. Instead of choosing a specific Genie Agent, the user just asks a question and Genie One routes it to the most relevant published Genie Agent(s) - using each Agent's description to decide the best match (which is why the description you set matters) - and returns the answer. It lives in the Databricks One business-user experience and is the "front door" your non-technical stakeholders use day-to-day. You will try it at the end of this lab. See Databricks One / Genie.

**Setting expectations**: Think of a Genie Agent as a brand new analyst, first job out of university. This analyst is going to be pretty good at writing SQL but will have no pre-existing knowledge about your business - not high level concepts, not low level jargon. The only knowledge this analyst will have about your business is the context provided. If context is vague or contains conflicting guidance, expect Genie to get confused when asked questions that contain business specific terminology, just like a human analyst would be. Furthermore, the Genie Agent will do its best to answer your question, looking for clues in the data that might help it understand what you mean when you use business terminology. Subsequently Genie may infer an interpretation which could be right, but could also be wrong. In this lab you may see this behaviour. Your job as a Genie Agent developer is to provide business context to maximise the chances that Genie will answer questions correctly.

---

1. Open **Genie Code** and type the following prompt:

   > Create a genie agent  based on the workspace.genie_demo.orders_enriched table.

2. Genie Code should create a new Genie Agent for you. Once done, you should be given the option of opening the Genie Agent via the Genie Code pane - when prompted click '**Allow**'. Alternatively you can click on '**Genie Agents**' in the left hand menu and then click on the newly created Genie Agent.
3. Close the Genie Code pane.
4. Your Genie Agent is ready to be used, noting its initial answers may not always be right - you will shortly start to fix that.

5. Under the chat box you will see some suggested sample questions. Feel free to click on one or more of these.

6. To ask your own question, click In the chat box (where it says '**ask your question …**') and type the following prompts and then click run ( ). Each subsequent prompt drills down into lower level details - simulating how a business user would typically interact with a Genie Agent

> How many orders are there, and what date range do they cover?

> What is the average order value per customer market segment? Show the result as a bar chart.

> What is the distribution of order priorities across different customer market segments?

> Give me a sum of the order value by month in Q2 of FY 1996.

Each time you ask a question you will see the Genie Agent execute an analysis step before it finally presents its response. You can expand the analysis by clicking on the down arrow on the '**Inspecting**' widget whilst it is running or the '**Analysis complete**' widget after it is finished. You should also see the option to '**Show code**' - click on this to see the SQL generated / executed to get the data it believes it needed to answer the input question.

7. The final question differs from the others because it asks the Genie Agent to interpret a business-specific term: "Q2 of FY 1996."

   The underlying dataset does not explicitly define when the financial year begins, and financial-year definitions vary between organisations. The answer must therefore reflect your organisation's definition of FY 1996. Without this context, the Genie Agent may:

   - Ask when your financial year begins so it can determine Q2.
   - Make its own assumption about the financial year (most likely financial year = calendar year meaning Q2 = 1 Apr - 30 Jun).

   This is a good example of why it is useful to review the Genie Agent's reasoning. Once the response has been generated, click '**Analysis complete**' directly beneath the question and review how the Agent interpreted FY 1996.

   Regardless of the response you received, we need to provide the Agent with the relevant business context to ensure future answers are grounded in your actual definition of FY. Databricks is expanding the ways this context can be supplied, but for a Genie Agent, the simplest approach is to add an instruction. You will do this shortly.

   Before moving on, take a mental note of the date range Genie Agent used as you will compare this to what you get post adding the instruction.

8. Click on '**Configure**' (top right of the page).
9. Directly below '**Configure**' you should see '**About**', '**Sources**', '**Instructions**' and '**Examples**' (with '**About**' in focus).
    - Under '**About**', you will find the Genie Agent description, owner, the SQL Warehouse used to process questions, and a set of common questions. Common questions are optional examples that help users understand what they can ask. They appear on the Agent's chat landing page. The initial examples shown here were generated by Genie Code. You can edit, add and remove common questions as needed.
    - Under '**Sources**' you will see the data assets this Genie Agent has access to (includes tables, SQL functions and what Databricks calls "Metric views"). All questions should be answered using the data listed here (subject to the users access rights). You should see workspace.genie_demo.orders_enriched table.
    - Under '**Instructions**' you will see a text box where you can add general instructions on how you want this Genie Agent to behave. We will define FY shortly.
    - Under '**Examples**' you have the ability to define the following:
        - **Sample queries** - example question-and-SQL pairs that teach Genie how to answer common questions and generate the correct SQL.
        - **Filters** - reusable boolean conditions that capture common business filtering logic (e.g. "high-value orders") so Genie applies them consistently.
        - **Measures** - governed definitions of KPIs / metrics (e.g. revenue, win rate) so Genie calculates aggregated values the same way every time.
        - **Fields** - custom attributes or per-row transformations (e.g. "deal size" buckets) that Genie can group and analyse by.
        - **Joins** - defined relationships between tables (which columns connect and the join type) so Genie builds correct JOINs.
    - You will also note at the very top "Monitor" and "Benchmark"
        - **Monitor** - lets you review the questions and responses users ask, see their feedback and any flagged answers, and track usage and accuracy over time to refine the agent.
        - **Benchmark** - a set of test questions (with expected SQL answers) you run to measure the agent's overall response accuracy as you tune it.

10. As you can see there are many Genie Agent capabilities that you can leverage to tune Genie Agent quality. In this lab you will apply just a couple, starting with a better description.

    Why do you need a better description? When a user asks a question in Genie One (which you will use at the end of the lab), the system routes it to the most relevant Genie Agent. If no matching Agent is found, it falls back to searching your data assets for the answer. Because Genie One relies heavily on Agent descriptions to make these matchmaking decisions, having a clear, keyword-rich profile is essential for discoverability. The default description is quite sparse - so let's upgrade it.

    With '**Configure**' still open, click '**About**'. Under the current description click on '**Edit**' and replace the contents of the description field with the following:

    Answers business questions about enriched wholesale-distribution orders, based on the workspace.genie_demo.orders_enriched table — TPC-H orders, joined to customer, nation and region so every order carries its customer's market segment, nation name and region name. Use this Agent for questions about order counts and the date range covered; order value, total order value and average order value broken down by customer market segment, nation, region or country; the distribution and mix of order priorities across market segments; and order value trends over time.

    Click '**Save**'.

11. Next you will define what your business means by "FY".

12. Assuming you still have '**Configure**' open, click on '**Instructions**' and add the following to the text box.

    > The business FY (Financial Year) starts Jul 1. For example FY 1996 starts Jul 1 1995.

13. Once you have added it, click '**Save'**.

14. Click on the '**Start a new chat**' button at the top of the page ( ) (this ensures the Genie Agent now uses your instruction). Ask the following question:

    > Give me a sum of the order value by month in Q2 of FY 1996.

15. This time the results should be based on Oct 1 - Dec 31 1995. Confirm Genie applies your definitions without being told / compare to the answer you previously got back.

16. So far all your questions have been answered using the Agent's 'Chat' mode. Genie Agents also support 'Agent' mode. Agent mode lets users get answers to complex, exploratory questions using multi-step reasoning and hypothesis testing. When you ask a question, Agent mode creates and refines a research plan, runs multiple SQL queries, learns from each result, and iterates until it has enough evidence to deliver a comprehensive report with citations, visualizations, and supporting tables.

17. In the chat interface, click on '**Agent**' and ask the following question (you may need to start a new chat to see the toggle):

> Compare the five market segments across order value, order volume, priority mix and growth over time, and tell me which segment is most attractive and why.

18. Observe the response in comparison to the previous responses, noting you now get he possible why instead of just the what.

19. Finally you will create some benchmarks. Benchmarks are a set of test questions you can run to assess Genie's overall response accuracy. For more information on using Benchmarks, see the documentation.

20. Whilst you can create Benchmarks manually, here you are asking Genie Code to build them for you. Open the **Genie Code** pane (not to be confused with the Genie Agent chat interface you have just been using) and type the following prompt:

> Build 6 benchmarks

21. It should start by gathering context about the workspace.genie_demo.orders_enriched table and then create relevant benchmark questions. Once finished close the Genie Code pane.

22. Click on '**Benchmarks**' (top), followed by '**Questions**', followed by '**Run all benchmarks**'.

23. For each question, Genie interprets the input, generates SQL, and returns results. The generated SQL and results are then compared against the SQL Answer defined in the benchmark question.

24. Open the **Genie Code** pane (again not to be confused with the **Genie Agent** chat interface you have just been using), click expand (top left in the Genie Code pane ( )), and type:

> Review this Genie Agent against its benchmark results, diagnose any questions it got wrong, and recommend improvements to its instructions and metadata.

25. Genie Code analyses the agent, proposes fixes (instructions, measures/filters, joins), and you can re-run the benchmark to see the score move. The idea here is to take the various recommendations, implement them and retest, repeating as necessary until you have achieved a response quality level you are happy with.

26. Try your Agent the way a business user would - via Genie One. So far you have talked to the Genie Agent directly. Now ask questions through Genie One, which sits across your Agents and picks the right one for you.

27. Open the app switcher (the grid / product-switcher icon at the top right) and select '**Databricks One**', Note: this is a rapidly evolving surface, so exact navigation may differ slightly.

28. Without naming an Agent, ask a couple of the same questions you asked in step 6 and confirm Genie One routes them to your orders_enriched Agent:

> What is the average order value per customer market segment?

> Give me a sum of the order value by month in Q2 of FY 1996.

29. All going well the financial-year question should be answered correctly (Oct 1 - Dec 31 1995) without you re-defining FY. Genie One answers using the same Genie Agent you configured, so it inherits the FY instruction you added earlier - the business context you provide is applied consistently, wherever the Agent is used.

30. While you are here, look at the dashboard: open the AI/BI Dashboard you published in Lab 3 via the Dashboards link on the left (it may also appear below the chat box) and confirm it is discoverable in the business-user experience too - the same governed data, surfaced both as a curated dashboard and as a conversational Genie Agent.

31. When you are done, open the app switcher again and switch back to the Lakehouse workspace via '**Lakehouse**' to continue.

---

# Lab 5: Exploratory Data Analysis

**Objective:** Use Genie Code to perform a full exploratory data analysis on a dataset, demonstrating Genie Code's ability to plan, execute, and iterate autonomously.

---

1. You can complete this lab entirely within the Genie Code Pane or in a Notebook.

   If you want to do this in a Notebook:

   **'+ New'** → **'Notebook'** ✅, Default language **'Python'** ✅, Genie Code pane open ✅, **'New Chat'** ✅

   If you want to do this entirely within the Genie Code Pane:

   **Genie Code** icon in the upper-right corner ✅. **Open Full Chat** icon in the top-left corner of the pane ✅

2. Enter the following prompt:

> Using python, perform an exploratory data analysis on workspace.genie_demo.orders_enriched. Include: row count, schema overview, summary statistics for numeric columns, the distribution of total price, order volume and average order value by month, a breakdown by customer market segment, and identify any data quality issues.

3. Watch as Genie Code plans the analysis steps / creates multiple notebook cells.
4. If you encounter any errors, follow the prompts to use Genie to diagnose the error and rerun revised code (this may be easier to do by clicking '**Accept**' in Genie Code and have it run the cells again for you - it should handle any errors via retry, repeating if necessary until it works).
5. If you haven't already, in the Genie Code pane click **Allow** to run all cells. It will re-execute the same cells which is ok before moving on to summarise its findings.
6. Now ask a follow-up question in the Genie Code pane:

> Based on the EDA, what are the most interesting patterns? Are there any outliers worth investigating?

7. Let Genie Code dig deeper into the patterns it identified (which is likely to include additional code cells - run these via the Genie Code pane when prompted).
8. Review the deeper analysis.

---

# What Next?

You have driven Genie Code through a no-code Lakeflow pipeline, an AI/BI dashboard, a Genie Agent and an exploratory data analysis. To keep building on what you have learned, here are some resources we recommend - all free.

## Keep Practising in Your Own Workspace

**Databricks Academy** - if the company you work for is an existing Databricks customer, then you should have access to free self-paced courses via your work email address. Good starting points after this lab: *Generative AI Fundamentals*, *Get Started with Databricks for Data Analysis*, *Get Started with Databricks for Data Engineering*, and *Get Started with Databricks for Machine Learning*.

## Go Deeper on the Topics in This Lab

- Genie Code Overview

- Genie Code for data science
- Extend Genie Code with agent skills
- AI functions on Databricks, ai_query reference
- What's new in Genie Code at Data + AI Summit 2026
- Pushing the Frontier for Data Agents with Genie
- The Start Learning courses in Free Edition (linked in the home page) - Databricks Fundamentals, Intro to Data Engineering and Intro to AI/BI
- Databricks on YouTube - product demos, conference talks, and how-to videos. Worth subscribing to for new feature drops.